

Delta-rs

Deep Dive Guide

Understanding the Rust Native
Delta Lake Library

Essence | Architecture | Philosophy | Usage

A1 English Level

Tank54 Training Materials

May 2025

Table of Contents

1. What is delta-rs?
2. Why Was delta-rs Created?
3. Philosophy and Principles
4. Architecture and Structure
5. Key Components
6. Operations You Can Do
7. How to Use delta-rs (Rust)
8. Python Bindings
9. Ecosystem and Integrations
10. delta-rs vs. Other Implementations
11. Tips for Developers

1. What is delta-rs?

delta-rs is a native Rust library that lets you read, write, and manage Delta Lake tables. It works without the Java Virtual Machine (JVM). Before delta-rs, the only way to use Delta Lake was through Apache Spark, which needs a heavy JVM setup. delta-rs changes this completely. It gives you a lightweight, fast, and safe way to work with Delta Lake tables using Rust.

The library is part of the official Delta Lake project under the Linux Foundation. It lives on GitHub at github.com/delta-io/delta-rs. You can use it directly in Rust projects, or through its Python bindings (the [deltalake](https://pypi.org/project/deltalake/) package on PyPI). This means both Rust developers and Python data engineers can benefit from it.

At its core, delta-rs uses Apache Arrow as its in-memory data format. Arrow is a cross-language standard for columnar data. Because delta-rs uses Arrow, it can work easily with many other tools like Polars, Pandas, DuckDB, and Apache DataFusion. This makes delta-rs a bridge between Delta Lake and the broader data ecosystem.

Key Point: delta-rs = Rust + Delta Lake + Apache Arrow. No JVM needed. Fast, safe, and lightweight.

What delta-rs gives you:

- Read Delta tables from local disk or cloud storage (S3, Azure, GCS)
- Write new data to Delta tables (append, overwrite)
- Merge, Update, Delete data with SQL-like operations
- Time travel - read older versions of your data
- Optimize - compact small files, Z-order for faster queries
- Vacuum - clean up old and unused data files
- Convert existing Parquet tables to Delta format
- SQL queries via Apache DataFusion integration

2. Why Was delta-rs Created?

delta-rs was started in April 2020 by R. Tyler Croy and QP Hou at Scribd, a digital content platform. They had a very practical problem. Their team used Delta Lake for big data processing with Apache Spark. But many daily tasks were small and simple - they did not need a full Spark cluster for those. Running Spark for small jobs was like using a big truck to deliver a single letter. It was too expensive and too slow to start.

Croy divided their workloads into three groups. First, big offline processing where Spark was the right tool. Second, small offline processing like Python or Ruby scripts that clean data or move data between systems. Third, boundary processing where online production systems need to read data from the data lake. For groups two and three, Spark was not the right choice. They needed something lighter.

The original proposal to Databricks said: "I do not want to burn down our current investment in Ruby for many of these workloads, not to mention retraining a number of developers to learn Scala or PySpark." This shows the real-world motivation behind the project. They wanted to use Delta Lake without changing their entire technology stack.

The main reasons for creating delta-rs:

- No JVM dependency - Avoid Java overhead for simple tasks
- Lower cost - No need for expensive Spark clusters for small jobs

- Language freedom - Work in Rust, Python, or any language with FFI
- Fast startup - Milliseconds instead of seconds (no JVM warm-up)
- Memory safety - Rust guarantees safety at compile time
- Ecosystem growth - Bring Delta Lake to non-Spark users

3. Philosophy and Principles

After five years of building delta-rs, R. Tyler Croy shared the lessons learned in a March 2025 blog post. These lessons reveal the core philosophy of the project. Understanding this philosophy helps you understand why delta-rs is designed the way it is.

3.1 Community is More Important Than Code

Open source is a social activity, not just a technical one. A welcoming community matters more than perfect code at the start. The delta-rs team focused on building a community where developers feel welcome. They merged pull requests even when the code was not perfect. This lowered the barrier for new contributors and helped the project grow fast. The result: more contributors, more features, and a healthier project overall.

3.2 Make It Easy to Contribute

The team treats community building like a sales process. They think about what motivates developers and how to attract them. The Python bindings (deltalake package) were the biggest game-changer. Many data engineers use Python, and having native Python support brought a wave of new contributors. The lesson: meet developers where they are, do not force them to come to you.

3.3 The Code Must Be Fun

Rust is a popular language and developers enjoy writing Rust code. This "fun factor" drove contributor velocity. For comparison, Apache DataFusion (a Rust project) had 82 monthly commit authors while Apache Spark had 70. Rust attracts developers who care about quality and performance. When the code is fun to write, people write more of it, and the project benefits from that energy.

3.4 Automate Everything

Fast release cycles create tight feedback loops with users. The delta-rs team invested heavily in automation: automated testing, automated releases, and continuous integration. They achieved 72% code coverage, which means most of the code is tested automatically. This allows them to release new versions quickly and with confidence. When a bug is found, it gets fixed and released fast.

3.5 Collaborate Broadly

The team works with users, contributors, and related projects. They collaborate with Databricks, the Apache Arrow community, DataFusion, Polars, DuckDB, and many others. This open collaboration approach has made delta-rs a central piece in the modern data ecosystem. No project can succeed alone, and delta-rs embraces this principle fully.

4. Architecture and Structure

delta-rs has a clean, layered architecture. At the top, you have the user-facing APIs (Rust and Python). In the middle, you have the core library logic. At the bottom, you have the foundation layer with Arrow, object_store, and Parquet. This layered design makes the code easy to understand and extend.

Layer Overview

Layer	Components	Purpose
User API	Rust crate, Python bindings (PyO3)	High-level interface for developers
Core Logic	deltalake-core, writer, delta-datafusion	Delta protocol, table state, operations
Foundation	Apache Arrow, object_store, parquet-rs	In-memory data, cloud storage, file format
Storage	S3, Azure, GCS, Local, HDFS, LakeFS	Where data lives (physical layer)

Table 1: Architecture layers of delta-rs

Crate Structure

The main `deltalake` crate is a meta-package. This means it re-exports from smaller sub-crates. This design lets you pick only the parts you need. The three main sub-crates are:

- `deltalake-core` - The heart of the library. It handles the Delta protocol: parsing transaction logs, managing table snapshots, handling schemas, and tracking file actions.
- `deltalake-writer` - Handles writing data to Delta tables. It converts Arrow RecordBatches into Parquet files and creates commit entries in the transaction log.
- `delta-datafusion` - Integration with Apache DataFusion. It registers Delta tables as DataFusion TableProviders, so you can run SQL queries directly against Delta tables.

Design Principle: The meta-package pattern lets you use only what you need. If you only want to read Delta tables, you can depend on `deltalake-core` alone. If you need SQL queries, add `delta-datafusion`.

```
USER-FACING APIs
[ Rust API ]           [ Python API (PyO3) ]
-----
CORE LOGIC
[ core: protocol, snapshot, schema, actions ]
[ writer: Parquet write, commit ]
[ datafusion: SQL query engine ]
-----
FOUNDATION LAYER
[ Apache Arrow ] [ object_store ] [ parquet-rs ]
-----
STORAGE BACKENDS
Local | S3 | Azure | GCS | HDFS | LakeFS
```

5. Key Components

5.1 Protocol Parser

The Delta Lake protocol uses a transaction log. Every change to a Delta table is recorded as a JSON commit file in the `_delta_log/` directory. The Protocol Parser reads these JSON files (and periodic Parquet checkpoints) to rebuild the complete state of the table at any version. This is how time travel works - you can reconstruct what the table looked like at version 1, version 5, or version 100.

5.2 Table Snapshot

A Table Snapshot is an immutable view of the table at a specific version. It contains the list of active data files, the table schema, partition columns, table properties, and protocol features. When you open a Delta table, you get its latest snapshot by default. You can also request a specific version for time travel. Snapshots are the key abstraction that makes Delta Lake transactions safe and consistent.

5.3 Actions

Every operation on a Delta table produces one or more Actions. Actions are the building blocks of the Delta protocol. The main action types are: Add (a new data file was added), Remove (a file was removed or changed), Metadata (table metadata like schema was updated), Protocol (protocol version changed), and CommitInfo (information about who made the commit and when). In delta-rs, these are represented as Rust enums, which makes them type-safe and efficient.

5.4 Storage Abstraction

delta-rs uses the `object_store` Rust crate to talk to storage systems. This crate provides a single, unified API for many storage backends. You write your code once, and it works with local files, AWS S3, Azure Blob, Google Cloud Storage, HDFS, and more. This abstraction is one of the most powerful features of delta-rs. Switching from local development to cloud production is as simple as changing the table URI from a local path to an S3 URL.

5.5 Writer

The Writer component converts Apache Arrow RecordBatches into Parquet files and creates the corresponding commit entries. It handles partitioning, statistics collection, and transaction log commits. The writer supports multiple write modes: append (add new data), overwrite (replace all data), and merge (upsert - update existing and insert new rows). All writes go through the Delta Lake transaction protocol, which ensures ACID guarantees.

6. Operations You Can Do

delta-rs supports a comprehensive set of Delta Lake operations. All of these are available in both the Rust API and the Python bindings. Here is a complete list of what you can do:

Operation	What It Does	Rust	Python
Create	Create a new Delta table	Yes	Yes
Read	Read data from a table	Yes	Yes
Append	Add new data to a table	Yes	Yes
Overwrite	Replace all data	Yes	Yes
Delete	Remove rows by condition	Yes	Yes
Update	Change values in a table	Yes	Yes
Merge	Upsert: update + insert	Yes	Yes
Vacuum	Remove old unused files	Yes	Yes
Optimize (Compact)	Merge small into big files	Yes	Yes
Optimize (Z-Order)	Cluster data for fast queries	Yes	Yes
Restore	Time travel to old version	Yes	Yes
History	View commit history	Yes	Yes
Convert to Delta	Upgrade Parquet to Delta	Yes	Yes
Add Constraint	Add CHECK constraint	Yes	Yes
FS Check	Remove corrupted files	Yes	Yes

Table 2: Supported operations in delta-rs

7. How to Use delta-rs (Rust)

7.1 Setup

First, add delta-rs to your Cargo.toml file. You also need Tokio for async runtime and Arrow for data handling:

```
[dependencies]
deltalake = "0.25"
tokio = { version = "1", features = ["full"] }
arrow = "54"
```

7.2 Open and Read a Table

Opening a Delta table is simple. You provide a URI (a local path or a cloud URL), and delta-rs loads the table metadata. You can also specify a version number for time travel:

```
use deltalake::DeltaTableBuilder;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Open table (latest version)
    let table = DeltaTableBuilder::
        from_uri("./data/my_table")
        .load().await?;

    // Get metadata
    println!("Version: {}", table.version());
    println!("Files: {:?}", table.get_file_uris()?);

    // Time travel to version 0
    let old = DeltaTableBuilder::
        from_uri("./data/my_table")
        .with_version(0)
        .load().await?;

    Ok(())
}
```

7.3 Write Data to a Table

Writing data uses Apache Arrow RecordBatches. You create your data as Arrow arrays, wrap them in a RecordBatch, and then use the writer to commit them to the Delta table:

```
use deltalake::{DeltaOps, DeltaTableBuilder};
use arrow::array::*;
use arrow::datatypes::*;
use arrow::record_batch::RecordBatch;
use std::sync::Arc;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let table = DeltaTableBuilder::
        from_uri("./data/my_table").load().await?;

    let schema = Arc::new(Schema::new(vec![
        Field::new("id", DataType::Int32, false),
        Field::new("name", DataType::Utf8, true),
    ]));

    let batch = RecordBatch::try_new(
        schema.clone(),
        vec![
            Arc::new(Int32Array::from(vec![1, 2, 3])),
            Arc::new(StringArray::from(
                vec!["Alice", "Bob", "Charlie"])),
        ],
    )?;

    // Write and commit
    DeltaOps(table).write(vec![batch]).await?;
    Ok(())
}
```

7.4 SQL Queries with DataFusion

If you want to run SQL queries on Delta tables, use Apache DataFusion. `delta-rs` provides first-class integration. Register the Delta table and run SQL:

```
use deltalake::DeltaTableBuilder;
use datafusion::prelude::*;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let ctx = SessionContext::new();
    let table = DeltaTableBuilder::
        from_uri("./data/sales").load().await?;

    // Register and query
    ctx.register_table(
        "sales", Arc::new(table.into())?;
    let df = ctx.sql(
        "SELECT region, SUM(amount) "
        "FROM sales GROUP BY region"
    ).await?;
    df.show().await?;
    Ok(())
}
```

8. Python Bindings

The Python package is called `deltalake` and it is available on PyPI. It is built using PyO3 and maturin, which allow Rust code to be called from Python with near-native speed. This is not a wrapper or a reimplementation - it is the same `delta-rs` core library exposed to Python. You get full Rust performance without writing any Rust code.

```
# Install
pip install deltalake

# Write data
import pandas as pd
from deltalake import DeltaTable, write_deltalake

df = pd.DataFrame({
    "id": [1, 2, 3],
    "name": ["Alice", "Bob", "Charlie"]
})
write_deltalake("./data/users", df)

# Read data
dt = DeltaTable("./data/users")
print(dt.to_pandas())

# Time travel
dt = DeltaTable("./data/users", version=0)

# Merge (upsert)
dt.merge(
    source_df,
    predicate="t.id = s.id",
    source_alias="s",
    target_alias="t"
).when_matched_update_all()
.when_not_matched_insert_all()
.execute()

# Optimize and Vacuum
dt.optimize.compact()
dt.optimize.z_order(["region"])
dt.vacuum()
```


Performance Tip: The Python `deltalake` package is 2-5x faster than PySpark for small to medium data operations because it has no JVM overhead. For large-scale ETL, Spark is still the better choice. Use the right tool for the job.

9. Ecosystem and Integrations

Because `delta-rs` uses Apache Arrow as its in-memory format, it connects easily with many tools in the data ecosystem. Arrow is the common language that allows different tools to share data without copying or converting. This is one of the biggest advantages of `delta-rs`.

Data Tools Integration

Tool	Language	Integration Type
pandas	Python	Direct read/write via Arrow
Polars	Rust + Python	Native Delta support
DuckDB	C++	Native extension via <code>kernel-rs</code>
Apache DataFusion	Rust	First-class SQL engine
Dask	Python	<code>dask-deltatable</code> connector
Daft	Rust + Python	Direct integration
Apache Ballista	Rust	Distributed compute
AWS awswrangler	Python	Delta Lake read/write
Ray	Python	<code>deltaray</code> (incubating)
Dagster	Python	IO manager integration

Table 3: Ecosystem integrations

Cloud Storage Support

All cloud storage backends are supported through the `object_store` Rust crate. You do not need to write storage-specific code. Just change the URI and `delta-rs` handles the rest. Supported backends include: AWS S3 (and compatible services like MinIO and Cloudflare R2), Azure Blob Storage and ADLS Gen2, Google Cloud Storage, local filesystem, HDFS, LakeFS, and Microsoft OneLake. This makes it easy to develop locally and deploy to the cloud.

Delta UniForm

An exciting new feature is UniForm (Universal Format). It allows Delta tables to be read as Apache Iceberg tables at the same time. This means you can write data once with Delta Lake and let Iceberg-based tools read it without any conversion. UniForm bridges the gap between the two major open table formats and gives you flexibility in your tool choices. `delta-rs` supports UniForm as part of the broader Delta Lake ecosystem.

10. delta-rs vs. Other Implementations

There are several ways to use Delta Lake. Understanding the differences helps you choose the right tool for your needs. The main options are: Delta on Spark (the original), Delta Standalone (JVM-based), `delta-rs` (Rust), and `delta-kernel-rs` (minimal Rust kernel).

Feature	<code>delta-rs</code>	Delta on Spark	<code>delta-kernel-rs</code>
Language	Rust + Python	Scala / Java	Rust + C FFI
JVM Needed	No	Yes	No
Memory Format	Apache Arrow	Spark Internal	Engine-defined

Feature	delta-rs	Delta on Spark	delta-kernel-rs
Query Engine	DataFusion	Spark SQL	Consumer picks
Write Support	Full	Full	Experimental
Python API	Native (fast)	PySpark (JVM)	Not direct
Startup Time	Milliseconds	Seconds	Milliseconds
Package Size	Small (~50MB)	Large (cluster)	Minimal
Best For	Lightweight ops	Big ETL	Connectors
Maturity	~5 years	~7 years	Newer

Table 4: Comparison of Delta Lake implementations

delta-rs vs. delta-kernel-rs: This is a common question. delta-rs is a full-featured library for developers who want to read, write, and manage Delta tables. delta-kernel-rs is a minimal, engine-agnostic kernel designed for building connectors for query engines (like DuckDB). delta-rs is tightly coupled with Arrow and DataFusion. delta-kernel-rs defines an Engine trait that consumers implement themselves, so it does not dictate an in-memory format. If you are building an application, use delta-rs. If you are building a query engine connector, look at delta-kernel-rs.

11. Tips for Developers

Here are practical tips to help you get the most out of delta-rs in your daily work. These tips come from the experience of the delta-rs community over five years of development.

Getting Started

- Start with Python, then learn Rust. The Python `deltalake` package is the easiest way to get started. Learn the Delta Lake concepts first, then move to Rust for more control and performance.
- Use `DeltaOps` for everything. The `DeltaOps` struct is the main entry point for all operations on a Delta table. It provides a fluent API that is easy to read and chain.
- Use `Arrow RecordBatches` directly. If you are working in Rust, learn `Arrow`. It is the standard in-memory format and many tools use it. Understanding `Arrow` helps you across the entire data ecosystem.

Performance

- Compact your tables regularly. Small files slow down queries. Use the `OPTIMIZE` operation to merge small files into larger ones. This is important after many appends.
- Use `Z-Order` for common filters. If you often filter on certain columns, use `Z-ORDER` to cluster data by those columns. This can speed up queries by 10x or more.
- Vacuum old files. After compacting or deleting, old files are still on disk. Use `VACUUM` to remove them and save storage costs.
- Use partitioning wisely. Partitioning by high-cardinality columns (like user ID) creates too many small files. Partition by low-cardinality columns (like date or region).

Development

- Run tests before contributing. delta-rs has 72% code coverage. When you add a feature, add tests for it. The CI pipeline checks your code automatically.
- Check protocol version support. Delta Lake has protocol versions (Writer V1-V7). Some features require newer versions. Check your table version before using advanced features.
- Follow the `arrow-streaming` pattern. When processing large datasets, stream `Arrow` batches instead of loading everything into memory at once. This keeps memory usage low.

- Use time travel for debugging. If something goes wrong, you can always go back to a previous version. Use the `.history()` method to see what changed and when.

Remember: delta-rs is built by a welcoming community. If you have questions, open a GitHub discussion. If you find a bug, open an issue. If you want to help, start with good-first-issues. The community will help you grow.